
Contents

Assemblatore, Linker e Loader	1
Introduzione	1
Processo di Traduzione	2
Passaggi principali:	2
Linguaggio Assemblativo	2
Definizione:	2
Caratteristiche:	2
Confusione terminologica:	2
Pseudo-Istruzioni	3
Caratteristiche:	3
Esempi nel RISC-V:	3
alt text	3
Direttive Assembly	3
Comandi principali:	3
Assemblatore	3
Funzione:	3
Moduli oggetto	4
Linker	4
Funzione:	4
Compiti principali:	4
Linking statico	5
Linking dinamico	5
Linking dinamico lazy(a runtime)	5
Loader	6
Binding di memoria	6

Assemblatore, Linker e Loader

Introduzione

Il documento analizza il processo di traduzione del codice sorgente in linguaggio macchina, passando per linguaggi intermedi come il linguaggio assemblativo. Esamina i ruoli di assemblatore, linker e loader, con esempi pratici e dettagli tecnici.

Processo di Traduzione

Passaggi principali:

1. Compilazione da C a linguaggio macchina:

- Il codice sorgente in C viene tradotto in linguaggio assembler (Livello 3).
- L'assembler converte il linguaggio assembler in linguaggio macchina (Livello 1).
- Il linker combina moduli oggetto per creare un eseguibile.
- Il loader carica l'eseguibile in memoria per l'esecuzione.

2. Esempio di traduzione:

- Funzione `scambia` in C tradotta in RISC-V Assembly.
 - Conversione in codice binario (ISA) con istruzioni come `slli`, `add`, `ld`, `sd`.
-

Linguaggio Assemblativo

Definizione:

- Linguaggio vicino al linguaggio macchina, con corrispondenza uno-a-uno tra istruzioni ISA e mnemoniche.

Caratteristiche:

- Uso di etichette simboliche.
- Primitive per allocazione di memoria.
- Definizione di macro.

Confusione terminologica:

- "Assembler" può indicare sia il programma traduttore che il linguaggio assemblativo.
 - "Linguaggio macchina" può riferirsi a istruzioni ISA o al linguaggio assemblativo.
-

Pseudo-Istruzioni

Caratteristiche:

- Alias per una o più istruzioni ISA che semplificano la programmazione.
- Non compaiono nel programma oggetto finale.

Esempi nel RISC-V:

- `nop`: Nessuna operazione.
- `li rd, immediate`: Carica un valore immediato.
- `mv rd, rs`: Copia un registro.

alt text

Direttive Assembly

Comandi principali:

- `.align`: Allineamento a potenze di 2.
- `.globl`: Definizione di simboli globali.
- `.section`: Specifica la sezione attiva (.text, .data, ecc.).
- `.macro` e `.endm`: Definizione e fine delle macro.

Assemblatore

Funzione:

Tradurre il linguaggio assembleativo in linguaggio macchina eseguibile.

1. Primo passo:

- Creazione della Symbol Table (etichette e posizioni).
- Gestione delle forward reference.

2. Secondo passo:

- Traduzione delle istruzioni mnemoniche in codice macchina

-
- Sostituzione dei simboli usando l'ILC

La symbol table è composta come:

Simboli	ILC	Altro
---------	-----	-------

Dove i simboli, sono i nomi delle etichette e ILC sono gli indirizzi che vengono dati ai simboli.

Gli indirizzi delle etichette(simboli), vengono salvati nei ILC che verranno poi sostituiti nel codice macchina prodotto. Ogni indirizzo dell'etichetta sarà distanziato da un'altra etichetta in base a quante istruzioni sono contenute nell'etichetta(4 byte per ogni istruzione).

Moduli oggetto

Sono l'output che l'assemblatore genera, è composto da:

- Identificatore, identifica l'oggetto, composto dal nome e dalla dimensione(quest'ultima ci serve per capire quanto leggere questo eseguibile e quanta ram allocare)
- Entry Point table, lista dei simboli esportati che gli altri moduli possono usare. Vengono identificati con il loro indirizzo
- External reference table, simboli importati da altri moduli e usati dall'oggetto. Vengono descritti dalla coppia: (funzione del modulo che richiede il simbolo esterno, nome del simbolo)
- Istruzioni macchina che verranno caricate ed eseguite dal processore
- Relocation dictionary, dizionario degli indirizzi che devono essere relocati dal linker
- End module, fine del modulo

Linker

Funzione:

Collegare moduli oggetto separati per creare un unico eseguibile.

Compiti principali:

- Fusione degli spazi di indirizzamento dei moduli oggetto.
- Aggiornamento degli indirizzi relativi e assoluti.

Ad esempio se nel modulo main chiamiamo un'altra etichetta di un altro modulo, il linker dovrà aggiornare gli indirizzi nel modulo main.

Il linker svolge le seguenti funzioni:

- Costruisce una tabella con i moduli oggetti e le loro dimensioni
- Assegna un indirizzo ad ogni modulo
- Trova tutte le istruzioni che accedono alla memoria e aggiunge a ciascun indirizzo una **relocation constant** corrispondente all'indirizzo di partenza del suo modulo così da far coincidere l'accesso in memoria con il suo modulo. Questa costante andrà usata nelle `lw`, `sw` come scostamento.
- Trova le istruzioni che chiamano altri moduli e aggiorna i loro indirizzi(ad esempio `jal`, ecc...)

Linking statico

Il linking statico permette di risolvere tutti gli indirizzi dei moduli durante il processo di linking, in questo modo tutti i file oggetto vengono aggiunti all'eseguibile senza bisogno di avere le librerie. Il file diventa più grosso ed occupa più memoria.

Linking dinamico

Il linking dinamico non risolve tutti gli indirizzi dei moduli esterni ma ciò viene demandato all'esecuzione del programma. Questo viene fatto dal sistema operativo che invoca un linker durante il caricamento del programma in memoria.

Linking dinamico lazy(a runtime)

Può succedere che le librerie vengono caricate a runtime in maniera condizionata(if, else). In questo modo possiamo risparmiare memoria caricando i moduli solo se necessario.

Se il linking è dinamico, nell'eseguibile viene prodotto un richiamo di funzione dummy che richiama il linker del sistema operativo che esegue il linking dinamico. Il linker dinamico può anche cambiare i dati nel segmento dati, cambiando l'indirizzo dummy, a quello della libreria. In questo modo dopo che viene eseguito il programma la prima volta, le successive volte non sarà necessario ripetere il processo di linking dinamico.

Il linking dinamico può dare problemi nel caso di funzioni deprecated che non sono più disponibili oppure librerie mancanti.

Loader

Il loader carica i programmi dal disco alla memoria centrale.

Il loader esegue i seguenti compiti:

- Legge l'intestazione dell'eseguibile
- Riservare uno spazio per contenerli
- Copiare il programma e dati nello spazio riservato
- Copiare nello stack i parametri passati al main
- Inizializzare i registri e lo stack pointer
- Saltare ad una procedura che copia i parametri dallo stack ai registri e invoca il main

Binding di memoria

La traduzione da etichette ad indirizzi assoluti avviene in diversi momenti, infatti se si spostano in memoria programmi che hanno già un indirizzo assoluto, ciò risulterebbe sbagliato.

Il binding viene fatto:

- Nell'assemblatore se conosce già gli indirizzi ILC delle istruzioni
- Al momento del linker quando chiamiamo moduli esterni (in questo caso viene fatto il linking statico perchè i simboli vengono messi direttamente nell'eseguibile)
- Al momento del loading (i moduli vengono collegati dinamicamente quando il programma viene eseguito. Linking dinamico)
- A runtime (possiamo scegliere di caricare moduli a runtime)